Trabajo Práctico Integrador

Bases de Datos 2

Tienda con MongoDB Atlas, API Express, frontend React y respaldo externo.

Informe técnico para revisión - 23 de octubre de 2026

Integrantes: Matías Yacob, Javier y Brenda Florencia Romero.

Repositorio: github.com/MatiasYacob/tpi-bases-datos-2.

Base de negocio: tpi_tienda. Clúster: TPI-Tienda. Proyecto Atlas: TPI-BD2.

Propósito y alcance

El sistema administra clientes, productos y pedidos de una tienda pequeña. Permite altas, consultas, modificaciones, cambios de estado, bajas lógicas y un reporte de ventas por categoría. Se trabaja con datos ficticios. Pagos, envíos reales, stock y autenticación de clientes quedan fuera del alcance.

La evidencia automática incorporada al repositorio corresponde a MongoDB temporal local, con Express y el driver oficial reales. La conexión y los flujos básicos contra Atlas fueron comprobados durante la implementación. No se confunden esas comprobaciones con un ensayo de rendimiento o recuperación en Atlas.

Arquitectura y responsabilidades

Capa	Tecnología y responsabilidad
Interfaz	React, TypeScript y Vite. Javier: pantallas y consumo de API.
API y modelo	Express, driver MongoDB y dotenv. Matías: validación, persistencia, índices y reporte.
Persistencia	MongoDB Atlas, base tpi_tienda; tres colecciones relacionadas.
Respaldo externo	Scripts Bash de Brenda para macOS; mongodump, launchd y mongorestore.

Flujo: navegador -> API HTTP local -> MongoDB Atlas. El respaldo se conecta a MongoDB por separado y no depende de Express ni de visitas a la aplicación.

Estado del documento

La implementación y sus pruebas locales están disponibles. La revisión independiente de Javier y el cierre del respaldo/restauración de Atlas por Brenda continúan pendientes de evidencia publicada. Este informe debe actualizarse con esos resultados antes de presentarlo como entrega final.

1. Modelo documental e integridad

Colecciones

Colección	Campos y relación
clientes	nombre, apellido, email, contacto embebido y campos de estado/fechas.
productos	nombre, descripción, categoría, precioCentavos y campos de estado/fechas.
pedidos	clienteId, copia histórica del cliente, estado, items embebidos y totalCentavos.

Cada documento utiliza _id de tipo ObjectId. La API devuelve id como string. Las fechas se almacenan como BSON Date y se serializan como ISO UTC. Los documentos de negocio incluyen activo, createdAt, updatedAt y deletedAt.

Embedding y linking

El contacto se embebe en el cliente porque se consulta junto con él. Los ítems se embeben en el pedido porque se leen como una unidad y están acotados a 50 productos distintos. Cada ítem copia nombre, categoría, precio unitario, cantidad y subtotal del momento de la compra.

clienteId e items.productoId enlazan entidades compartidas que cambian de forma independiente. Además de los identificadores, el pedido guarda una copia histórica del cliente y los productos. Una edición posterior del catálogo no reescribe ventas anteriores.

Validaciones y decisiones

El backend valida campos admitidos, referencias activas, cantidades y estados. Los validadores $jsonSchema de MongoDB comprueban estructura, tipos y categorías. Las reglas de transición y la existencia de referencias corresponden al backend.

El email se normaliza a minúsculas, sin espacios exteriores, y tiene índice único. Un email de cliente inactivo sigue reservado. Los importes se representan en centavos enteros; ARS 1500,50 se guarda como 150050. Se validan enteros seguros y cálculos individuales de pedidos.

Baja lógica

DELETE aplica $set: activo=false, deletedAt y updatedAt. Las lecturas normales y las modificaciones filtran activo=true. No se eliminan físicamente clientes, productos ni pedidos desde el CRUD. Las pruebas consultan MongoDB después de la baja y verifican que los documentos y las fechas siguen presentes.

2. API, pedidos y reporte

Contrato HTTP

Base local: http://localhost:3000/api. La API es una demostración local sin autenticación HTTP. Las credenciales de Atlas quedan en backend/.env; nunca se incluyen en variables VITE_.

Operación	Rutas o formato
Listar y crear	GET / POST en /clientes, /productos y /pedidos.
Detalle, edición, baja	GET / PATCH / DELETE en cada recurso /:id.
Listados	data y paginacion con pagina, limite y total. Orden createdAt e _id descendentes.
Filtros	Productos por categoría; pedidos por cliente, estado e intervalo de fechas.
Errores	error con codigo, mensaje y campo opcional. HTTP 400, 404, 409 o error interno.
Salud	GET /health: 200 con MongoDB disponible; 503 si no responde.

POST devuelve 201. PATCH pedido permite solo estado. Los campos desconocidos o reservados se rechazan. El contrato ampliado incluye solicitudes y respuestas reales del ensayo local; sus identificadores no se reutilizan contra Atlas.

Creación y estados del pedido

El cliente envía clienteId e ítems con productoId/cantidad. El servidor verifica referencias activas, lee precios actuales, genera instantáneas y calcula subtotales y total. No acepta precios ni total enviados por el navegador. Las cantidades admitidas son enteros de 1 a 100 por producto.

Transiciones: pendiente -> confirmado/cancelado; confirmado -> completado/cancelado. Completado y cancelado son terminales. Repetir un estado devuelve 409. La escritura incluye el estado previo en el filtro para proteger cambios concurrentes. La lectura del catálogo y el alta no son una transacción global.

Agregación de ventas

GET /reportes/ventas-por-categoria selecciona pedidos activos y completados por createdAt. Pipeline: $match -> $unwind items -> $group -> $project -> $sort. Agrupa por categoría histórica y suma items.cantidad e items.subtotalCentavos.

Desde es inclusivo y hasta exclusivo, con fechas ISO que incluyan zona. La interfaz convierte el día final a la medianoche siguiente. No se suma totalCentavos después de unwind: se duplicaría por cada ítem.

Limitación conocida: sumas agregadas extremadamente grandes pueden exceder el entero seguro de JavaScript. El caso límite está identificado y pendiente de corrección; no afecta los importes de los ejemplos de esta evidencia.

3. Índices y evidencia explain

Método

Se ejecutó npm run evidence:indexes sobre MongoDB 7.0.24 temporal local: 100 clientes, 10000 productos y 20000 pedidos ficticios deterministas. Se utilizaron filtros de los endpoints y los índices definidos por el proyecto.

Antes de crear los índices secundarios solo existía _id_. Después se ejecutó initialize(db). No se forzó hint. Para cada consulta se comprobó que los mismos diez ids aparecieran en el mismo orden. No se alteraron índices ni datos de Atlas.

Consulta	Docs antes	Docs después	Claves después
Productos activos por categoría	10000	10	10
Pedidos activos generales	20000	10	10
Pedidos por cliente, estado y fecha	20000	10	10
Control: productos sin categoría	10000	10	9000

Interpretación

En las primeras tres consultas el plan pasó de COLLSCAN con SORT a IXSCAN con FETCH y LIMIT. Las igualdades del filtro fijan el prefijo del índice y los campos temporales/id permiten obtener el orden sin una etapa SORT explícita.

El índice de productos es activo/categoria/createdAt/_id. El de pedidos generales es activo/createdAt/_id. El de pedidos filtrados es clienteId/activo/estado/createdAt/_id. El índice único de clientes sobre email también protege integridad; se comprobó el rechazo de duplicado con código 11000.

El control sin categoría mantiene SORT y recorre 9000 claves. El campo categoría intermedio impide usar directamente ese índice para el orden global por fecha. El caso muestra que un índice no beneficia por igual a todas las variantes.

Los milisegundos de una sola ejecución local están afectados por caché y selección de plan. No son un benchmark de Atlas. La evidencia compara documentos, claves, etapas y resultados; además, todo índice tiene costo de almacenamiento y mantenimiento.

Evidencia reproducible

docs/evidencias/indices/explain.json conserva las consultas, planes y executionStats. El README de esa carpeta explica el ensayo. Se reproduce desde backend con npm ci y npm run evidence:indexes. Requiere dependencias de desarrollo y puede descargar el ejecutable oficial de MongoDB.

4. Pruebas de integración

Entorno y resultados

El comando npm run evidence:integration ejecutó 48 solicitudes HTTP mediante Supertest hacia Express y cuatro comprobaciones directas usando el driver oficial sobre MongoDB temporal real. Las colecciones tenían los validadores e índices del proyecto. Todas las comprobaciones pasaron.

No se usaron mocks de repositorios, Atlas ni el navegador para producir esta evidencia. El proceso no lee .env y cierra su MongoDB temporal al terminar. Las fechas se fijan dentro del setup aislado para verificar límites exactos.

Grupo	Comprobación realizada
Clientes	Alta, email normalizado/duplicado, contacto opcional y reemplazo en PATCH.
Productos	Precio en centavos, categorías válidas, filtros y páginas sin duplicados.
Pedidos	Rechazo de total/precio del navegador, productos repetidos y cantidades fuera de rango.
Historial	Cambio del catálogo sin modificar nombres ni precios del pedido.
Estados	Transiciones válidas y rechazo de saltos, repetición y cambios terminales.
Reporte	Dos categorías sin duplicar total; desde inclusivo y hasta exclusivo.
Baja lógica	Inactivos ocultos, referencias rechazadas e historial conservado.
MongoDB	Documentos siguen presentes tras DELETE; esquema rechaza estructura inválida.

Ejemplo verificado

Se creó Mouse a 150000 centavos y Cuaderno a 20000. El pedido de dos Mouse y un Cuaderno devolvió totalCentavos=320000. Luego Mouse pasó a 190000 y se cambió el nombre del cliente: el detalle del pedido mantuvo el precio y cliente históricos.

Al completar el pedido, el reporte devolvió tecnología=300000 y librería=20000. Dar de baja cliente/producto no eliminó ese historial. Dar de baja el pedido sí lo excluyó del reporte.

Evidencias y pruebas complementarias

docs/evidencias/integracion/http.json guarda método, ruta, cuerpo enviado, código esperado y cuerpo real para cada solicitud. docs/pruebas-integracion.md resume resultados y reproducción. Los ids y fechas pertenecen a la ejecución temporal.

npm test suma diez pruebas aprobadas: cuatro de API/MongoDB temporal y seis regresiones de restauración con respuestas de DB simuladas. Incluyen JSON malformado, salud, concurrencia y seed repetible. No se presenta la ejecución del autor como revisión independiente de Javier.

5. Respaldo y recuperación

Implementación publicada por Brenda

Los scripts Bash para macOS separan el respaldo de la aplicación web. backup.sh invoca mongodump mediante configuración privada, crea una copia temporal, valida BSON/metadatos de las tres colecciones y publica resguardos_tpi/AAAA-MM-DD solo después de una exportación exitosa.

El bloqueo impide ejecuciones simultáneas. La opción --forzar valida una nueva copia antes de reemplazar la del día. La retención conserva cinco fechas completas y se ejecuta después de publicar una copia nueva. Se documentaron pruebas locales de retención, fallo de conexión y conservación de copias previas.

programar-backup.sh utiliza launchd. instalar-macos.sh prepara una carpeta de ejecución fuera del Escritorio, conserva la configuración privada y compara los scripts instalados. La documentación registra pruebas por horario y registro sin duplicados, con la tarea pausada al finalizar.

Restauración y verificación

restaurar-prueba.sh restaura a tpi_tienda_restore_test en MongoDB local y rechaza un destino que ya contenga colecciones. El verificador compara clientes, productos y pedidos con conteos-esperados.json de esa copia.

El respaldo todavía no genera automáticamente ese archivo de conteos: la documentación indica prepararlo manualmente sin escrituras concurrentes y asociarlo a la copia correcta. Una copia forzada requiere actualizar esos conteos.

Evidencia disponible y límites

Según docs/respaldo.md, la restauración local inicial recuperó tres documentos y midió un segundo hasta finalizar las consultas. Esa muestra no representa el RTO de Atlas. El RPO objetivo de hasta unas 24 horas depende de que las tareas diarias realmente se ejecuten; aumenta con fallos o equipo apagado.

La última documentación publicada deja pendiente el ensayo de respaldo contra Atlas y el flujo completo actualizado de restauración con conteos. El PR #5 corrigió la validación: primero compara conteos y luego acepta pedidos activos, inactivos o cero pedidos. Pasaron seis regresiones; falta repetir la restauración real en macOS con esa versión.

Estos puntos deben confirmarse con Brenda antes de cerrar el informe. No se afirma que el respaldo de Atlas ni una tarea nocturna activa estén demostrados. mongodump es un volcado lógico BSON; para una prueba coherente deben pausarse escrituras y registrarse esa condición.

6. Ejecución, revisión y cierre

Reproducción del sistema

Paso	Comando o condición
Backend	Node 24; npm ci dentro de backend.
Configuración	backend/.env privado con URI y base tpi_tienda.
Preparación inicial	npm run db:init y npm run seed solo al preparar la base y con autorización.
API	npm start en backend; escucha localmente en 127.0.0.1:3000.
Frontend	npm ci y npm run dev en frontend; abrir localhost:5173.
URL pública del cliente	VITE_API_URL=http://localhost:3000/api; no contiene secretos.
Evidencia aislada	npm test, npm run evidence:integration y npm run evidence:indexes.

Revisión cruzada

Javier debe ejecutar los comandos desde una copia actualizada, registrar versiones, commit, resultados y observaciones propias. La ficha docs/revision-cruzada.md incluye el recorrido, preguntas de defensa y un espacio para su veredicto. Estado actual: pendiente de ejecución por el revisor.

No es suficiente que el autor muestre un resultado anterior. Si el revisor encuentra un fallo, debe registrar cómo reproducirlo y acordar su corrección antes de aprobar el módulo. La prueba funcional contra Atlas se coordina para no alterar datos ajenos.

Conclusiones

El sistema implementa el flujo cliente/producto/pedido con reglas visibles en el backend, referencias compartidas e instantáneas históricas. La baja lógica conserva datos y el reporte utiliza valores históricos por ítem. Las pruebas aisladas verifican errores y resultados sobre persistencia real.

La comparación de índices muestra mejoras concretas de acceso para tres consultas y una limitación del catálogo sin categoría. Las métricas están documentadas sin extrapolar tiempos a la nube. El cierre grupal requiere agregar la revisión independiente y los resultados finales de respaldo y recuperación.

Entrega pendiente

Antes de presentar: cerrar la evidencia de Brenda, incorporar capturas reales sanitizadas si las requiere la cátedra, registrar revisión cruzada, confirmar requisitos/fecha en campus y preparar el ZIP sin .env, credenciales, node_modules ni volcados BSON.

Fuentes internas y referencia técnica

Contrato: docs/contrato-api.md. Pruebas: docs/pruebas-integracion.md.

Índices: docs/evidencias/indices/. Respaldo: docs/respaldo.md.

MongoDB explain: mongodb.com/docs/v7.0/reference/explain-results/.